```
ocr_detection/
 ├── requirements.txt          <-- Package dependencies
|
 ├── src/
|    ├── utils.py             <-- Module 1: Helper tools (validation, CLAHE, deskew & box drawing)
|    ├── ocr_engine.py         <-- Module 2: Core AI engine (CRAFT text detection & CRNN character recognition)
|    └── app.py               <-- Module 3: Streamlit interactive web dashboard
|
└── tests/
    ├── test_images/          <-- Sample verification benchmark photos (plates, receipts, documents)
```

```
User (Browser)        src/app.py          src/utils.py          src/ocr_engine.py
    |                    |                    |                      |
    |--- 1. Upload file -->|                  |                      |
    |                    |--- 2. Validate bytes>|                     |
    |                    |<-- 3. True / False --|                     |
    |                    |                                           |
    |                    |--- 4. Convert to BGR Matrix                |
    |                    |                                           |
    |                    |--- 5. detect_and_extract_text(image, thresh)--->|
    |                    |                    |                      |
    |                    |                    |<-- a. preprocess_image --|
    |                    |                    |      (CLAHE, bilateral, |
    |                    |                    |       auto-deskew)     |
    |                    |                    |--- b. clean_image ------>|
    |                    |                                           |-- c. CRAFT text detector
    |                    |                                           |-- d. CRNN sequence recognizer
    |                    |                                           |-- e. disambiguate_license_plate
    |                    |                                           |-- f. sort reading order
    |                    |                    |<-- g. draw_boxes --------|
    |                    |                    |      (boundary clamped) |
    |                    |                    |--- h. annotated_image -->|
    |                    |                                           |
    |                    |<-- 6. Returns structured results dict ----------|
    |                    |      {annotated_image, total_text_blocks, ...} |
    |                    |                                           |
    |                    |--- 7. render_metrics(total, confidence, latency)|
```

| |--- 8. format_export_data(detections, 'csv') |

|<-- 9. Display image -| |

Purpose: Verifies that uploaded raw bytes form a valid, uncorrupt image file.

Purpose: Normalizes lighting, eliminates vehicle headlight glare, removes paper shadows, and rotates skewed documents horizontally.

Algorithm:

1. CLAHE: Converts image to LAB color space, applies CLAHE (clipLimit=2.0, tileGridSize=(8, 8)) on L-channel to balance contrast, merges back to BGR.

2. Bilateral Filtering: Denoises background paper grain while preserving sharp character edge gradients (d=5, sigmaColor=50, sigmaSpace=50).

Purpose: Draws high-contrast bounding boxes around detected text blocks with adaptive background banners displaying text snippets and confidence percentages.

Purpose: Initializes the CRAFT text detection and CRNN sequence recognition model weights into memory.

Purpose: Solves classic alphanumeric OCR confusion on vehicle license plates.

1. image: np.ndarray → The uploaded photo represented as a pixel matrix.

Step 1 (Preprocess & Deskew): Invokes preprocess_document_image() to normalize contrast, denoise, and straighten tilt.

Step 2 (Localize & Recognize Characters): Runs CRAFT text detector to locate character regions, then runs CRNN deep sequence model to read words.

Step 3 (Disambiguate & Sort): Applies disambiguate_license_plate() to correct vehicle plate numbers, and sorts text blocks in natural reading order (top-to-bottom, left-to-right).


```
{
  # 1. The image array with the bounding boxes and text badges painted on it
  "annotated_image": np.ndarray,

  # 2. Total number of text blocks found in the photo (e.g., 6)
  "total_text_blocks": 6,

  # 3. Complete extracted plain text string
  "extracted_text": "PURCHASE ORDER #89201\nVendor: Global Logistical Supplies\n...",

  # 4. Detailed line-by-line detections list
  "detections": [
    {
      "line_id": 1,
      "text": "PURCHASE ORDER #89201",
      "confidence": 0.892,
      "box": [[40, 70], [450, 70], [450, 110], [40, 110]]
    },
    {
      "line_id": 2,
      "text": "Vendor: Global Logistical Supplies",
      "confidence": 0.704,
```

```
        "box": [[40, 120], [380, 120], [380, 155], [40, 155]]
      }
    ]
```

- Takes 'annotated_image' → displays side-by-side with original input in the right column.
- Takes 'total_text_blocks' → displays on Metric Card 1: 'Text Blocks Detected: 6'.
- Calculates average confidence → displays on Metric Card 2: 'Average Confidence: 84.5%'.
- Measures processing latency → displays on Metric Card 3: 'Processing Latency: 640 ms'.

3.1 Layout & State Management

- Page Setup: st.set_page_config(page_title='Universal OCR & Plate Detection System', layout='wide').
- Sidebar Controls:
  - App Mode Selection: Tabs between 1-Click Benchmark Demos, Custom File Upload, and Live Camera Snapshot.
  - Confidence Slider: st.sidebar.slider('Detection Confidence Threshold', 0.10, 0.95, 0.30, 0.05).
  - Engine Status Badge: Displays AI Engine: Deep Learning (CRAFT + CRNN).
  - Privacy Pledge: 100% On-Premise Privacy badge.

3.2 Photo Upload Workflow

1. User drops file into st.file_uploader(type=['jpg', 'jpeg', 'png', 'bmp']).
2. Calls validate_image_bytes(bytes). If invalid, displays clear warning message.
3. Decodes image and invokes detect_and_extract_text(image, confidence_threshold).
4. Renders summary metrics via st.columns(3): Text Blocks Detected, Average Confidence, Latency.
5. Displays original vs. annotated images side-by-side and provides 1-Click Download buttons for TXT, CSV, and JSON.

3.3 Live Camera Workflow

1. Activates st.camera_input('Capture snapshot from camera').
2. Automatically runs OCR pipeline on snapshot frame and displays instantaneous text extraction and bounding boxes.

3.4 Launch Command:

4.1 Purpose of the Test Suite

The test suite guarantees that all utility functions, AI character recognition pipelines, edge cases, and real-world defect images function correctly before client release. It runs via: pytest tests/ -v.

4.2 Test Data Fixtures (tests/test_images/)

To test the model with real-world inputs, curated defect benchmark test images are stored in tests/test_images/:

- **tests/test_images/1_car_plate_motion_blur.jpg: Vehicle license plate with horizontal motion blur (KA 03 MG 4589).**
- **tests/test_images/2_receipt_shadow_defect.jpg: Thermal store receipt with harsh diagonal shadow gradient (ASPIRIN, TOTAL DUE).**
- **tests/test_images/3_document_skewed_tilted.jpg: Business purchase order tilted by 15° (PURCHASE ORDER #89201).**
- **tests/test_images/4_signboard_perspective_blur.jpg: Warning signboard with 3D perspective distortion (ROAD WORK AHEAD).**
- **tests/test_images/5_car_plate_glare_defect.jpg: Vehicle license plate with bright headlight glare reflection (DL 01 AB 78).**

Test Function	What It Tests	Pass / Assert Condition
test_validate_image_bytes_valid()	Upload validation with real image bytes.	assert validate_image_bytes(valid_bytes) is True
test_validate_image_bytes_corrupt()	Upload validation with empty or corrupt bytes.	assert validate_image_bytes(b'bad data') is False
test_validate_image_bytes_oversized()	Rejection of payloads > 200MB (DoS defense).	assert validate_image_bytes(oversized) is False
test_estimate_skew_angle_straight()	Verifies flat image has near-zero skew angle.	assert abs(angle) < 5.0
test_preprocess_document_image_rgb()	Contrast and bilateral filtering on 3-channel images.	assert processed.shape[2] == 3 and not np.isnan(processed).any()
test_draw_ocr_bounding_boxes_clamping()	Margin clamping for text at (0, 0) and (W, H).	Asserts banner coordinates clamped within (0, W) and (0, H).
test_format_export_data_csv()	CSV table structure and column headers.	assert 'Line_ID,Text,Confidence,Bounding_Box' in csv_str
test_load_ocr_engine_returns_wrapper()	Deep learning engine initialization.	assert engine.engine_type == 'deep_learning'
test_disambiguate_license_plate_digits()	Converts letter 'O' to '0' and 'I' to '1' in digit slots.	assert disambiguate_license_plate('KAO3MG4589') == 'KA 03 MG 4589'
test_ocr_car_plate_motion_blur()	Real detection on 1_car_plate_motion_blur.jpg.	assert 'KA' in text and '4589' in text
test_ocr_receipt_shadow_defect()	Real detection on 2_receipt_shadow_defect.jpg.	assert 'PHARMACY' in text and 'TOTAL' in text
test_ocr_document_skewed_tilted()	Real detection on 3_document_skewed_tilted.jpg.	assert 'PURCHASE ORDER' in text and '89201' in text
test_ocr_signboard_perspective_blur()	Real detection on 4_signboard_perspective_blur.jpg.	assert 'ROAD' in text or 'WORK' in text

test_ocr_blank_white_image()	Edge case: blank white photo with zero text.	assert total_text_blocks == 0 without crashing

4.4 Visual Verification Artifact Generation:

When test_ocr_engine.py and benchmark runners execute, they write visual output images to an output/ directory:

• output/result_1_car_plate_motion_blur.jpg: Image showing the plate enclosed in high-contrast box labeled KA 03 MG 4589.

• output/result_2_receipt_shadow_defect.jpg: Image showing store receipt line items extracted through shadow.

• output/result_3_document_skewed_tilted.jpg: Image showing straightened purchase order with all 6 lines extracted.

• output/edge_cases/: Visual artifacts verifying salt-and-pepper noise, wide banners, and inverted dark mode.

Edge Case Scenario	Potential Failure Point	System Defense Mechanism
Empty or 0-byte file uploaded	**Crash during image decode**	**validate_image_bytes checks file length and returns clean error message.**
Corrupted / Truncated image	**Unhandled PIL/OpenCV exception**	**Caught in two-phase PIL validation (verify() + load()) without breaking session.**
Oversized payload (> 200 MB)	**Host memory starvation / DoS**	**Hard 200MB limit halts processing before memory allocation.**
Photo with 0 text characters	**Loop index out-of-range**	**Function returns total_text_blocks = 0, original image displayed intact with banner: 'No text detected.'**
Text tilted up to 45°	**OCR fragmentation / misreads**	**estimate_skew_angle() computes contour angle; rotate_image() counter-rotates frame straight.**
Headlight glare on vehicle plate	**Blinding character washout**	**CLAHE equalizes L-channel histogram in LAB color space, restoring character contrast.**
Text touching image boundary	**Coordinate negative out-of-bounds**	**Bounding badge coordinates are clamped: startX = max(0, startX).**

PRD Feature	Requirement Description	Implementing Function / File
Feature 1 & 2	**Locate text blocks and read alphanumeric characters**	**src/ocr_engine.py → detect_and_extract_text()**
Feature 3	**Vehicle plate alphanumeric disambiguation (0 vs O)**	**src/ocr_engine.py → disambiguate_license_plate()**
Feature 4	**Auto-deskew tilted text up to 45°**	**src/utils.py → estimate_skew_angle(), rotate_image()**
Feature 5	**CLAHE shadow and glare normalization**	**src/utils.py → preprocess_document_image()**
Feature 6	**Draw bounding boxes with boundary clamping**	**src/utils.py → draw_ocr_bounding_boxes()**
Feature 7	**1-Click pre-loaded benchmark demos**	**src/app.py → Tab 1 (generate_synthetic_demo_image())**

Feature 8	**Drag & drop photo uploader**	**src/app.py → Tab 2 (st.file_uploader())**
Feature 9	**Live webcam capture**	**src/app.py → Tab 3 (st.camera_input())**
Feature 10	**Confidence sensitivity slider**	**src/app.py → st.sidebar.slider()**
Feature 11	**Summary metrics cards (Blocks, Confidence, Latency)**	**src/app.py → st.columns(3)**
Feature 12	**Download annotated results (TXT, CSV, JSON)**	**src/utils.py → format_export_data() & st.download_button()**
Quality Gate	**Automated Verification on real images**	**tests/test_ocr_engine.py → pytest**